CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

Project – Fundamental Topics

A MOBILE PERSONAL FINANCE MANAGEMENT APPLICATION SUPPORTED BY A LARGE LANGUAGE MODEL — PERFIN

Major:	Software Engineering
Cohort:	49
Course class:	CT239H M01
Advisor:	Dr. Phan Phuong Lan
Student:	Nguyen Thanh Trong
Student ID:	B2305615

Semester 3, academic year 2025–2026

Can Tho, 2026

ABSTRACT

Context: Manual entry makes personal-finance histories incomplete or inconsistent, while figures generated by an LLM cannot be written directly to a ledger. **Objective:** This project builds PERFIN, a mobile prototype that accepts natural transaction input while preserving verifiable data and analysis. **Method:** Text, receipt images and speech are converted into typed drafts and written to PostgreSQL only after confirmation. Deterministic functions compute balances, trends, anomalies, cash flow, budgets and goals; the LLM only extracts intent and narrates facts. **Results:** The backend suite passes 182/182 tests, and the local parser obtains macro-F1 0.177 on 5,265 transactions. On the same 63-sentence sample, the parser and Gemini obtain 0.204 and 0.607. Correction retrieval reaches 68.19% on sentences preselected because the parser was wrong; the 0% baseline follows from this sampling rule and is not representative of the whole system. **Significance:** The architecture uses an LLM without replacing the data-and-algorithm core. OCR/STT accuracy and production readiness have not been established.

Keywords: personal finance management, large language model, data analysis, PostgreSQL, verifiable systems.

Contents

List of Tables

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BOM	Byte Order Mark
CER	Character Error Rate
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DB	Database
ERD	Entity–Relationship Diagram
F1	Harmonic mean of precision and recall
FK	Foreign Key
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
IQR	Interquartile Range
JSON	JavaScript Object Notation
KV	Key–Value
LLM	Large Language Model
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
OLS	Ordinary Least Squares
PDF	Portable Document Format
PERFIN	Personal Finance — the name of the prototype presented in this report
PK	Primary Key
REST	Representational State Transfer
SQL	Structured Query Language
STT	Speech-to-Text
SUS	System Usability Scale
TC	Test Case — a test-case identifier at the specification level
TTL	Time To Live
UAT	User Acceptance Testing

Abbreviation	Meaning
UI	User Interface
UML	Unified Modeling Language
VND	Vietnamese Dong
WER	Word Error Rate

STATUS AND RESULT CONVENTIONS

This report clearly distinguishes implemented components, measured results and behavior that exists only at the design level. This convention prevents expectations or smoke tests from being presented as completed quality evidence.

Table 1: Convention for levels of information confidence in the report

Label	Meaning	Usage
Implemented	A corresponding component exists in source code, migrations or tests.	Proves existence; does not automatically prove quality.
Measured	A command, timestamp and reproducible run result exist.	May be used to report figures in the results section.
Target	A desired threshold used as an acceptance criterion.	Must not be presented as an achieved result.
Design target	Expected behavior after in-scope fixes are completed.	Must be tested before being converted to “measured”.
Out of scope	Not a goal of the current project.	Mentioned only under limitations or future work.

CHAPTER 1: INTRODUCTION

1.1. PROBLEM STATEMENT

Personal finance management requires regular recording, consistent classification and an understanding of income–expense history. Financial literacy is directly associated with long-term planning and decision-making [1]. Conventional forms, however, require many steps, so omissions or category errors propagate into balances, budgets and reports.

Natural input such as “ăn phở sáng nay 45k bằng Momo” reduces effort but creates new risks. A system must understand Vietnamese currency notation, relative dates, multiple transactions in one sentence, approximate wallet names and classification context. Receipt images and speech introduce additional OCR/STT error. Writing an inferred result directly to the database can therefore corrupt all downstream analysis.

PERFIN follows one principle: PostgreSQL and deterministic functions are the source of truth; LLM, OCR and STT services only produce intermediate data for review. The project addresses three research questions:

1. How can text, image and speech be transformed into structured transactions without compromising data correctness?
2. Which deterministic algorithms are suitable for explainable insights from personal-finance history?
3. Where should an LLM participate to improve natural interaction without taking over computation or modifying data autonomously?

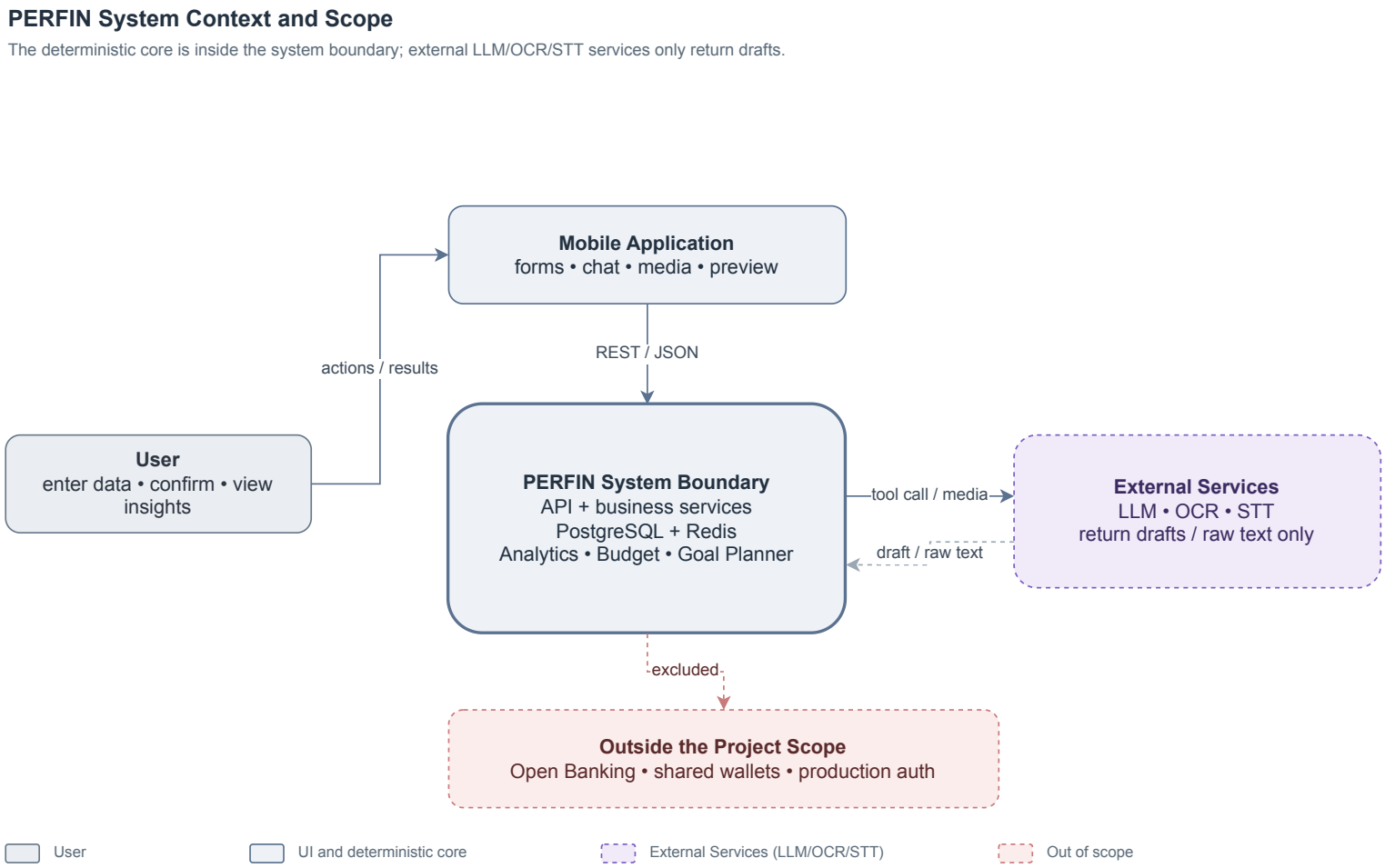

Figure 1: PERFIN system context and scope; LLM, OCR and STT services remain outside the trusted data boundary.

Figure 1 limits the project to acquiring, organizing and analysing personal-finance data. Bank connectivity, shared wallets, production authentication and high-availability infrastructure are out of scope.

1.2. OBJECTIVES

The overall objective is to complete, within 13 weeks, a personal-finance prototype in which structured data and deterministic algorithms form the foundation and the LLM is a controlled language-understanding and narration layer.

Table 2: Specific objectives, acceptance criteria and deadlines

Code	Specific objective	Evaluation criterion	Deadline
O1	Build a financial data model with keys, constraints and atomic updates.	Migrations, ERD and rollback tests; no partial write in critical flows.	Week 5
O2	Build text, image and speech extraction with clarification, preview and confirmation.	No database write when fields are missing; retain source and intermediate data.	Week 8
O3	Implement trend, anomaly, runway, recurring, correlation, budget and goal algorithms.	Normal, boundary and invariant cases agree with hand-computed results.	Week 9
O4	Restrict the LLM to intent understanding, tool calling and fact narration.	Tool schema, validation, fallback and confirmation remain separate from write authority.	Week 10
O5	Evaluate the prototype with reproducible tests and experiments.	Report the exact scope of unit, integration and smoke evidence and disclose missing measurements.	Week 13

The criteria in Table 2 are planned acceptance conditions. A result is called "measured" only when its input, environment and artifact are available.

1.3. RESEARCH METHODS

The project combines four methods:

1. **Literature review:** synthesize relational data, text similarity, explainable statistics, OCR, STT and function calling to establish the LLM boundary.

2. **Requirements and systems analysis:** study the input–confirmation–analysis process and model actors, data, constraints and ledger-corruption failure modes.
3. **Prototype design:** develop in short iterations; separate route–service–model responsibilities and implement formulas as independently testable deterministic functions.
4. **Experimentation:** use unit, integration and system smoke tests plus labelled datasets to compare the parser with the LLM, measure correction retrieval and analyse errors. Media smoke tests demonstrate execution only, not accuracy.

1.4. IMPLEMENTATION PLAN

The plan starts on 11 May 2026 and spans 13 weeks. Testing and report writing may overlap, but each week has a verifiable deliverable.

Table 3: Thirteen-week implementation plan

Week	Dates	Main work	Deliverable
1	11–17 May	Review the problem, guideline and related applications.	Research questions and scope.
2	18–24 May	Elicit requirements and normalize the SRS.	FR/NFR list and overall use case.
3	25–31 May	Design architecture and the LLM boundary.	Architecture diagram and module contracts.
4	01–07 Jun	Design the domain and relational schema.	Class diagram, ERD and migration draft.
5	08–14 Jun	Implement ledger, constraints and transactions.	O1 and core-data tests.
6	15–21 Jun	Implement parser, normalization and fuzzy matching.	Text pipeline and quality gate.
7	22–28 Jun	Integrate LLM, clarification and pending state.	Tool schema and preview/confirm flow.
8	29 Jun–05 Jul	Integrate OCR/STT through adapters.	Media pipeline and provider-error handling.
9	06–12 Jul	Implement analytics, budget and goal planning.	O3 and formula unit tests.

Week	Dates	Main work	Deliverable
10	13–19 Jul	Complete narration, feedback and worker logic.	O4, fallback and side-effect controls.
11	20–26 Jul	Run integration tests, import data and conduct experiments.	Logs, result JSON and error analysis.
12	27 Jul–02 Aug	Restructure the report and normalize diagrams.	Guideline-compliant LaTeX v2.
13	03–09 Aug	Review, compile, repair references and prepare the demo.	Submission PDF, source and user guide.

1.5. PROJECT SCOPE

The implementation includes income and expense transactions, wallet transfers, categories, budgets, recurring bills, goals, data analysis and prototype-level text/image/speech input. Evaluation focuses on the data model, transformation correctness and algorithms. The mobile interface captures input and demonstrates the workflow.

Open Banking, foundation-model training, shared wallets, investment advice, production authentication and authorization, high availability and uptime commitments are excluded. The prototype uses `default_user`; user foreign keys indicate an extension path, not evidence of secure multi-user operation.

1.6. CONTRIBUTIONS

The principal contributions are a constrained data model with atomic updates; a multimodal human-in-the-loop pipeline; independently testable analytics; an LLM boundary limited to extraction and narration; and an evaluation protocol that separates algorithm quality, provider execution and system readiness.

1.7. REPORT STRUCTURE

Chapter 1 presents the problem, objectives, methods and plan. Chapter 2 summarizes theory, compares alternatives and explains technology choices. Chapter 3 presents the SRS, design, detailed algorithms and testing. Chapter 4 reviews the objectives, limitations and future work.

CHAPTER 2: THEORETICAL BACKGROUND

This chapter summarizes the concepts, models and technologies used by the project. Project-specific formulas and thresholds are moved to the detailed design in Chapter 3.

2.1. FINANCIAL DATA AND PREPROCESSING

2.1.1. Relational Ledger and Data Integrity

A financial transaction minimally contains an amount, type, date, description, category and wallet. Income increases a balance, expense decreases it, and a wallet transfer creates two opposite changes that must not be counted as income or expense. Multi-record changes belong in one database transaction so they either complete or roll back together.

A relational database provides foreign keys, decimal types, domain constraints and aggregate queries. Durable business data must remain separate from short-lived clarification, preview, cache and queue state. Daily, weekly and monthly series must preserve the calendar axis and zero-fill inactive periods; dropping zero periods distorts slopes, burn rate and correlation.

2.1.2. Normalization and Text Similarity

Input is normalized for case, punctuation, whitespace, currency units and relative dates before schema mapping. For strings a, b, Levenshtein similarity [2] is

$$s_{edit} = 1 - \frac{D_{lev}(a, b)}{\max(|a|, |b|)}. \tag{2.1}$$

For token sets T_a, T_b, the Dice coefficient [3] is

$$s_{dice} = \frac{2|T_a \cap T_b|}{|T_a| + |T_b|}. \tag{2.2}$$

Levenshtein distance captures nearby typing errors, whereas Dice is useful when phrase order or components differ. PERFIN combines exact matches, aliases, both scores and a safety margin between the two best candidates. The chosen thresholds are detailed in Section 3.2.3.2.

2.2. EXPLAINABLE ANALYTICAL TECHNIQUES

2.2.1. Trend, Anomaly and Correlation

Ordinary least squares (OLS) [4] models a time series as $\hat{y}_i = a + bx_i$. Its slope is

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \tag{2.3}$$

The sign of b gives direction and R^2 gives linear fit. OLS is explainable and testable but does not model seasonality or future events. A z-score compares an observation with the mean in standard-deviation units, while the interquartile range is more robust to extremes [5]. PERFIN uses both as review signals, not as fraud conclusions.

Pearson's coefficient [6] measures linear co-movement:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{2.4}$$

Correlation is interpreted only within the observed window and does not imply causation.

2.2.2. Runway, Recurring Items, Budgets and Goals

Runway extrapolates how long the current balance can support the recent spending rate. Recurring-item discovery groups transactions by description, amount and date interval. Budget forecasting uses spending pace over elapsed days, while goal planning uses the remaining amount, deadline, contribution and debt simulation. These techniques prioritize transparent assumptions and boundary handling. PERFIN's formulas and windows appear in Section 3.2.3.3.

Table 4: Selected analytical techniques

Technique	Role	Reason for selection
OLS	Trend and short-horizon forecast.	Few parameters; slope and fit are explainable.
Z-score + IQR	Unusual amounts or days.	Combines distribution sensitivity with quantile robustness.
Runway	Cash-depletion warning.	Direct calculation with explicit assumptions.
Rule-based grouping	Recurring-spending discovery.	Suits small data and requires no training.
Pearson	Co-moving category pairs.	Standardized coefficient with clear interpretation.
Budget/Goal planner	Limits, forecasts and progress.	Hand-checkable results and what-if simulation.

2.3. OCR, STT AND LANGUAGE MODELS

2.3.1. OCR and Speech-to-Text

OCR generally includes image preprocessing, text-region detection, recognition and post-processing; Tesseract is a representative architecture [7]. STT converts audio to a transcript, and Whisper illustrates recognition learned from large-scale data [8]. In PERFIN, both services only return raw text and provider metadata. The result follows the same normalization, preview and confirmation pipeline as manual input.

Quality requires ground truth. CER/WER measure text error, while transaction-field accuracy measures the effect on amount, date, category and line items. A smoke test on a few files establishes provider execution, not accuracy.

2.3.2. Local Parser and LLM

Transformers use attention to model token relationships and underpin modern LLMs [9]. Multimodal models such as Gemini extend this ability to text and media [10]. Structured output constrains shape but cannot prove that an amount is correct, a category belongs to the user or an operation is authorized.

Table 5: Comparison of the local parser and an LLM

Criterion	Local parser	LLM via function calling
Language coverage	Strong on known patterns; weak on free-form input.	Better at variations and context.
Reproducibility	High, offline and no API cost.	Model/provider dependent and slower.
Control	Rules and failures are traceable.	Requires locked schema, validation and fallback.
PERFIN role	Baseline, fallback and quality gate.	Primary extraction/routing when configured.
Business authority	Returns a typed draft; never writes directly.	Proposes a tool and arguments; never writes directly.

PERFIN uses provider-schema function calling [11]. The LLM proposes a tool and arguments; the backend validates, builds a preview and waits for confirmation. For insight narration, the LLM only receives precomputed facts. This preserves interaction benefits without making the model a source of truth.

2.4. TECHNOLOGIES AND RATIONALE

Table 6: Technologies, roles and alternatives

Technology	Reason for selection	Limitation or alternative
React Native + Expo	One codebase for forms, chat, image, audio and mobile preview.	Separate native apps are only needed for device-specific requirements.
Node.js + Express	Fits I/O-oriented APIs and shares JavaScript with business functions and tests.	A modular monolith is sufficient; microservices add operations cost.
PostgreSQL [12]	Foreign keys, `DECIMAL`, transactions and aggregates fit a ledger.	File/NoSQL designs provide weaker constraints and rollback.
Redis [13]	TTL for pending state, clarification, cache and queue data.	Never the system of record; memory fallback is development-only.
BullMQ [14]	Scheduling, retry and job IDs for idempotent work.	Basic cron lacks equivalent queue and retry controls.

Technology	Reason for selection	Limitation or alternative
Gemini + local parser	Combines language coverage with a testable fallback.	Provider adapters prevent business-logic lock-in.
Local or cloud OCR/STT	PaddleOCR/PhoWhisper favour privacy; cloud providers remain replaceable.	Every raw output still follows validation and preview.

2.5. RELATED APPLICATIONS AND SELECTED DIRECTION

Money Lover [15] and MISA MoneyKeeper [16] represent form- and dashboard-oriented finance applications. They are strong at structured records but require many entry steps. Chatbots reduce entry work yet risk untraceable numbers; specialist analytics are often difficult for general users.

PERFIN does not attempt commercial feature completeness. It targets the intersection of a relational ledger, deterministic algorithms, human confirmation and a bounded language layer. This direction fits the project's data-and-algorithm focus and allows each part to be evaluated independently.

Table 7: Research gaps and PERFIN responses

Gap	Risk	Selected response
Natural input may be miswritten	Dirty data propagates to reports	Typed draft, validation, preview and confirmation.
Media provides raw text only	Wrong amount, date or category	Ground truth, field accuracy and confirmation.
Chatbot numbers lack provenance	Hallucination and weak auditability	SQL/analytics creates facts; narrator only explains.
Classification does not adapt	Repeated errors	Thresholded, controlled correction retrieval.
Retry duplicates effects	Corrupt data or duplicate alerts	Job IDs, fingerprints and idempotent operations.

CHAPTER 3: APPLICATION RESULTS

3.1. SOFTWARE REQUIREMENTS SPECIFICATION

3.1.1. Overall Description

PERFIN is a mobile prototype for entering, normalizing, managing and analysing personal-finance data. Chat and media complement forms as input channels. The backend performs every financial calculation, constraint check and data change; LLM, OCR and STT services have no direct PostgreSQL access and are not authoritative numeric sources. FR/NFR identifiers and test traceability follow the intent of IEEE 830 [17].

Table 8: Product context and constraints

Item	Description
User	A general user records and interprets personal-finance data without requiring statistical expertise.
Environment	React Native/Expo on mobile or web, Express API and PostgreSQL; Redis and a worker are configuration-dependent.
System of record	PostgreSQL stores business data; Redis stores only TTL-bound pending, clarification, cache and queue state.
Constraints	The demo uses `default_user`, has no production authentication/authorization and defaults to VND.
Dependencies	LLM/OCR/STT providers are replaceable; failure must produce a real error or fallback, never fabricated data.
Assumption	The user reviews previews; guidance does not replace professional financial advice.

Table 9: Actors and concerns

Actor	Role	Concern or limitation
User	Enters, edits, confirms, manages and views reports.	Language/media-derived data must be reviewed before storage.
External AI service	Returns a typed draft, OCR text or transcript.	Cannot execute business operations or write the database.
Background worker	Runs reminders, analysis and file cleanup.	Jobs require user scope, retry and duplicate-effect protection.
Developer administrator	Configures environment, migrations and providers.	Not a business actor of the application.

Shared rules are: business amounts must be finite and valid; transaction dates cannot be in the future; wallets and categories belong to the current profile; negative wallet balances are allowed; chat/media operations require preview and confirmation; multi-record updates are atomic; reports exclude soft-deleted transactions by default; and insufficient data yields a warning or `null`, never an invented value.

3.1.2. Functional Requirements

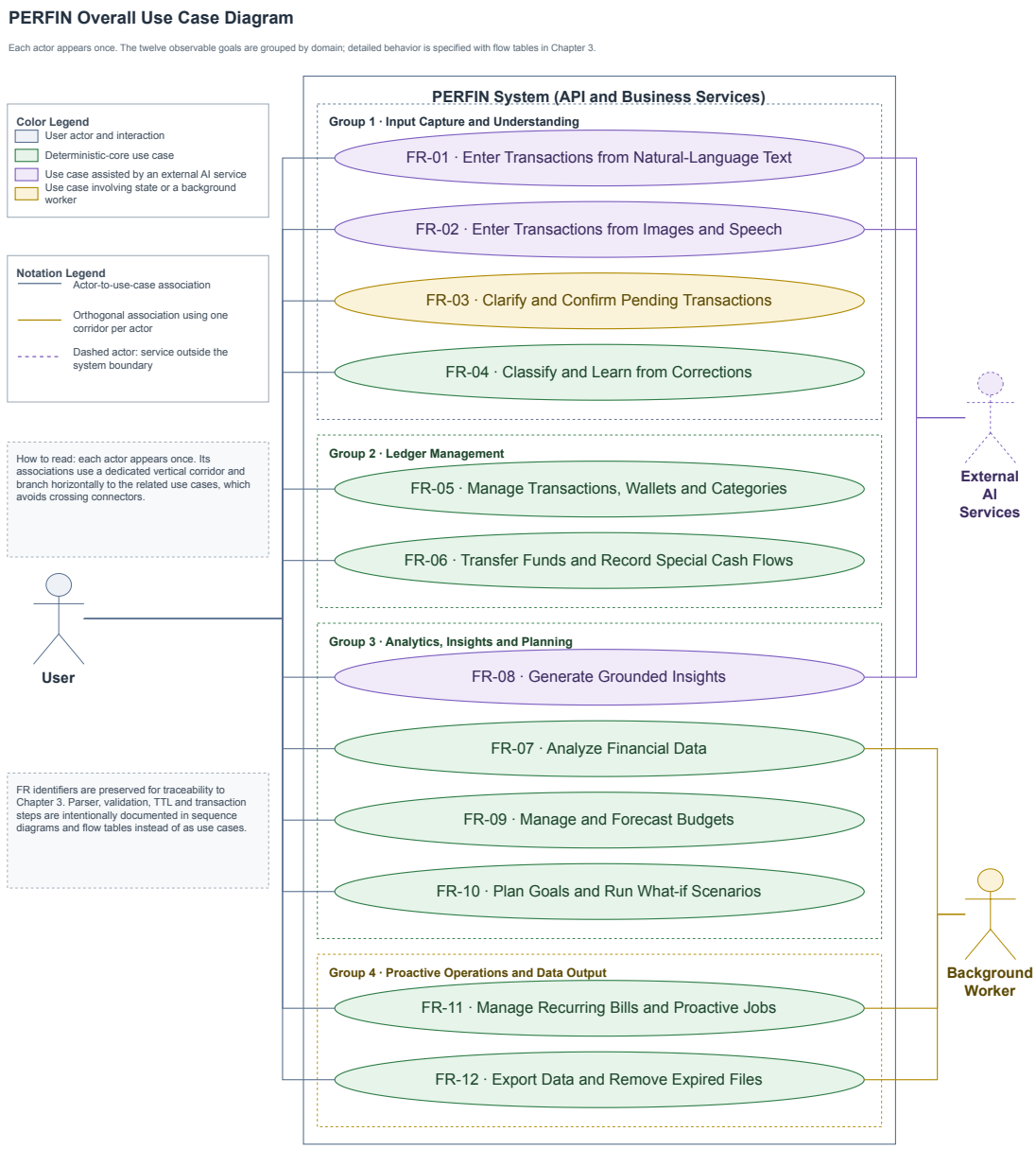

Figure 2: Overall PERFIN use case diagram. Feature details are specified with flow tables instead of twelve separate diagrams.

Figure 2 contains observable actor goals only. Parser, validation, transaction, TTL and retry behaviour belongs in the flow tables or the sequence/flow diagrams in Section 3.2.3.

Table 10: FR-01 — Enter Transactions from Text

Item	**Description**
Feature name	Enter transactions from natural-language text.
ID	FR-01.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user needs fast entry; the AI Orchestrator returns a typed draft; Transaction Service protects the ledger.
Summary	Convert one sentence into one or more structured transactions and create a preview before writing.
Relationships	Includes FR-03 and FR-04; uses FR-05 after confirmation.
Normal flow	**Step 1:** User submits text. **Step 2:** Extract and normalize. **Step 3:** Validate schema, category and wallet. **Sub 1:** Build preview. **Step 4:** User confirms. **Step 5:** Commit atomically and return the result.
Subflows	**Sub 1 — Preview:** (1) Store a TTL-bound draft; (2) show amount, type, date, category, wallet and warnings.
Alternate flows	**Step 2:** Provider failure uses the local parser or returns an error. **Step 3:** Missing/ambiguous fields invoke FR-03. **Step 4:** Edit or cancel causes no write.

Table 11: FR-02 — Enter Transactions from Image or Speech

Item	**Description**
Feature name	Enter a transaction from a receipt image or speech.
ID	FR-02.
User	User; External AI service.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs visible raw text; adapters report the provider and failures; the backend rejects invalid files.

Item	Description
Summary	Convert image/audio into reviewable text, then reuse FR-01.
Relationships	Includes FR-01 and FR-03.
Normal flow	**Step 1:** Select a valid file. **Step 2:** Backend invokes OCR or STT. **Step 3:** Display raw text and provider. **Sub 1:** User edits/confirms text. **Step 4:** Continue with FR-01.
Subflows	**Sub 1 — Media review:** (1) edit a voice transcript; (2) for a receipt, select total or individual items when present.
Alternate flows	Invalid type or size above 10 MB is rejected. A provider that returns no text produces an error and no mock or pending draft.

Table 12: FR-03 — Clarify and Confirm Pending Transactions

Item	Description
Feature name	Clarification, preview and pending-transaction confirmation.
ID	FR-03.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user can edit/cancel; the KV store applies TTL; a preview can be consumed once.
Summary	Collect missing fields across turns and prevent inferred data from being written.
Relationships	Used by FR-01, FR-02, FR-09, FR-10 and FR-12.
Normal flow	Store intent and missing fields; ask for the next field; merge and revalidate; build/edit a preview; atomically claim pending state; call the business service.
Subflows	**Preview:** attach a `pending_id` with five-minute TTL and update only with a valid payload.
Alternate flows	Continue asking while fields are missing. Expired, incorrect or claimed IDs cannot write; cancellation clears state.

Table 13: FR-04 — Classify and Learn from Corrections

Item	Description
Feature name	Category classification and correction retrieval.
ID	FR-04.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The matcher must be explainable; the user corrects labels; feedback failure must not corrupt a committed transaction.
Summary	Select a category by exact/alias/fuzzy/LLM methods and reuse confirmed corrections.
Relationships	Extends FR-01 and may be triggered by an FR-05 category edit.
Normal flow	Normalize description; load same-profile categories/corrections; rank candidates; apply a strong correction; return category, confidence and match kind; store feedback after confirmation.
Subflows	Select at most five similar corrections and use them as context or a controlled override.
Alternate flows	Low score, small margin or conflicting corrections return Other/request a selection. Feedback-write failure is logged only.

Table 14: FR-05 — Manage Financial Data

Item	Description
Feature name	Manage transactions, wallets and categories.
ID	FR-05.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user needs CRUD and restore; services maintain balance and history consistency.
Summary	Create, view, edit, soft-delete and restore transactions and manage profile-scoped categories/wallets.
Relationships	Used by FR-01/FR-02 and supplies FR-07–FR-12.

Item	Description
Normal flow	Select operation; validate payload and ownership; compute balance delta; update transaction and wallet atomically; return record and new balance.
Subflows	Write the transaction, update the wallet, commit, then invalidate cache best-effort.
Alternate flows	Out-of-scope IDs or invalid payloads are rejected. A negative balance remains valid. Any pre-commit failure rolls back.

Table 15: FR-06 — Transfer Funds and Record Special Cash Flows

Item	Description
Feature name	Wallet transfer and investment gain/loss.
ID	FR-06.
User	User.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	One user action must update both wallets and history consistently.
Summary	Record internal transfers without changing total assets and classify investment cash flow correctly.
Relationships	Uses FR-05 wallet and transaction entities.
Normal flow	Enter source, target, amount and date; validate distinct wallets; lock in stable order; debit, credit and write history; commit.
Subflows	In one transaction, subtract source, add target and write `wallet_transfers`; net change equals zero.
Alternate flows	Same/missing/out-of-scope wallets or invalid amount/date roll back. A future negative source balance alone is not rejected.

Table 16: FR-07 — Analyse Financial Data

Item	Description
Feature name	Trend, anomaly, runway, recurring and correlation analysis.

Item	Description
ID	FR-07.
User	User; Background worker.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	Analytics needs clean data; users need period, sample size and method.
Summary	Compute deterministic facts over non-deleted data and zero-filled time axes.
Relationships	Reads FR-05 and supplies FR-08 and FR-11.
Normal flow	Select analysis and period; query profile data; zero-fill and check sample count; run the algorithm; return facts, method and warnings.
Subflows	Run OLS, z-score/IQR, runway, recurring or Pearson through one result contract.
Alternate flows	Sparse data or zero variance returns a degraded result or `null`; no causal claim or fabricated transaction.

Table 17: FR-08 — Generate Grounded Insights

Item	Description
Feature name	Grounded insight and persona narration.
ID	FR-08.
User	User; External AI service.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs clear explanation; Analytics owns facts; the narrator cannot alter numbers.
Summary	Explain precomputed facts and return them with the narration for traceability.
Relationships	Includes FR-07 and may use a persona when consent is enabled.
Normal flow	Receive question/report request; generate facts; load permitted persona; narrate facts; validate response contract; return facts and narration.
Subflows	Cache facts with TTL and store narration metadata without treating it as authoritative.

Item	Description
Alternate flows	Sparse facts return a warning. Provider failure returns a deterministic template. Numeric-check failure falls back instead of emitting the response.

Table 18: FR-09 — Manage and Forecast Budgets

Item	Description
Feature name	Monthly budgets, recommendations and forecasts.
ID	FR-09.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	Users need explicit limits; Budget Service needs consistent period and category data.
Summary	Manage limits, forecast month end and propose budgets from history.
Relationships	Reads FR-05 and may invoke FR-03 from chat.
Normal flow	Select month; query spending; compute progress and forecast; produce warning/recommendation; confirm before saving a recommendation.
Subflows	Baseline uses historical mean plus buffer; 50/30/20 and hybrid constrain total allocation.
Alternate flows	No history yields a setup request, not a prediction. Duplicate category-period updates atomically.

Table 19: FR-10 — Plan Goals and What-if Scenarios

Item	Description
Feature name	Savings, purchase and debt-payoff goal planning.
ID	FR-10.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user needs formula-backed progress; Goal Planner must expose infeasible assumptions.

Item	Description
Summary	Calculate contribution, duration, amortization and what-if plans without modifying the original until confirmation.
Relationships	Reads FR-05/FR-07 and may use FR-03.
Normal flow	Enter goal; validate amount/date/rate; compute allocatable cash; run goal-specific formula; return plan and warnings; save after confirmation.
Subflows	What-if reruns the same pure function with an additional contribution and keeps the original plan unchanged.
Alternate flows	Invalid inputs request correction. Payment below first-month interest warns of negative amortization. Cancel causes no write.

Table 20: FR-11 — Manage Recurring Bills and Proactive Jobs

Item	Description
Feature name	Recurring bills, payments and proactive messages.
ID	FR-11.
User	User; Background worker.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs correct due dates; the worker needs retry and idempotency; PostgreSQL must prevent duplicate effects.
Summary	Manage schedules, record one payment per period and generate grounded reminders/alerts.
Relationships	Reads FR-05/FR-07; may call FR-08 for wording only.
Normal flow	Scheduler enqueues a user-scoped job; worker loads due data; computes deterministic facts; persists an event by unique key; advances schedule after success.
Subflows	Payment writes payment history, transaction and next due date atomically.
Alternate flows	Duplicate event/job returns the existing result. Transient failure retries with backoff. Redis failure stops the worker while the interactive API may continue.

Table 21: FR-12 — Export Data and Remove Expired Files

Item	Description
Feature name	Data export, export history and expired-file cleanup.
ID	FR-12.
User	User; Background worker.
Necessity	Optional.
Classification	Medium.
Participants and concerns	The user needs correct data; the exporter needs safe escaping; cleanup cannot remove valid files.
Summary	Generate filtered CSV, store history and delete files after TTL.
Relationships	Reads FR-05, may use FR-03 from chat, and delegates cleanup to the worker.
Normal flow	Select format/period; validate and query profile data; create a safe file; write `export_history`; return a download link.
Subflows	Escape formula-like cells, write UTF-8 BOM and row count, then assign TTL.
Alternate flows	Empty data returns a header-only file. The path once labelled PDF only creates HTML and is not claimed as a true PDF export.

3.1.3. Non-functional Requirements

Table 22: Non-functional requirements and acceptance methods

ID	Group	Testable requirement	Measurement
NFR-01	Recovery	Provider/Redis failure fabricates no data; restore demo service and backup data within three hours after an incident.	Fault injection, restart and restore on a test database.
NFR-02	Integrity	No partial create, update, delete, restore, transfer or payment; credentials are never transmitted or logged in clear text.	Rollback tests and configuration/log scanning; TLS for network deployment.

ID	Group	Testable requirement	Measurement
NFR-03	UI	Vietnamese blue–white interface, consistent navigation and no overflow at 390×844.	UI smoke on Dashboard, Transaction and Chat.
NFR-04	Timing	Acknowledge within three seconds; target text p95 ≤ 3 s and media p95 ≤ 8 s in a disclosed environment.	At least 30 runs per flow with provider, hardware, cache and error rate.
NFR-05	Correctness	Algorithms match expected normal/boundary cases; negative balances are valid; narrator invents no number.	Unit tests, invariants and facts–response checker.
NFR-06	Privacy	Remove temporary media; use traits only with consent; ID queries include user scope.	Temporary-file, consent and cross-scope tests.
NFR-07	Worker consistency	Retrying an event/job creates no duplicate payment, message or export.	Repeat fingerprints and count effects.
NFR-08	Traceability	Analysis includes period, sample size, method, facts and degradation status.	Contract tests and FR–test–artifact matrix.

Recovery, latency, OCR/STT accuracy and complete numeric grounding remain acceptance targets; unmeasured targets are not presented as achieved results.

3.2. SOFTWARE DESIGN

3.2.1. Architecture

PERFIN is a modular monolith: domain services are separated by responsibility while the API remains one process; a separate worker handles background jobs. This matches project scale, reduces operating cost and preserves independent module tests.

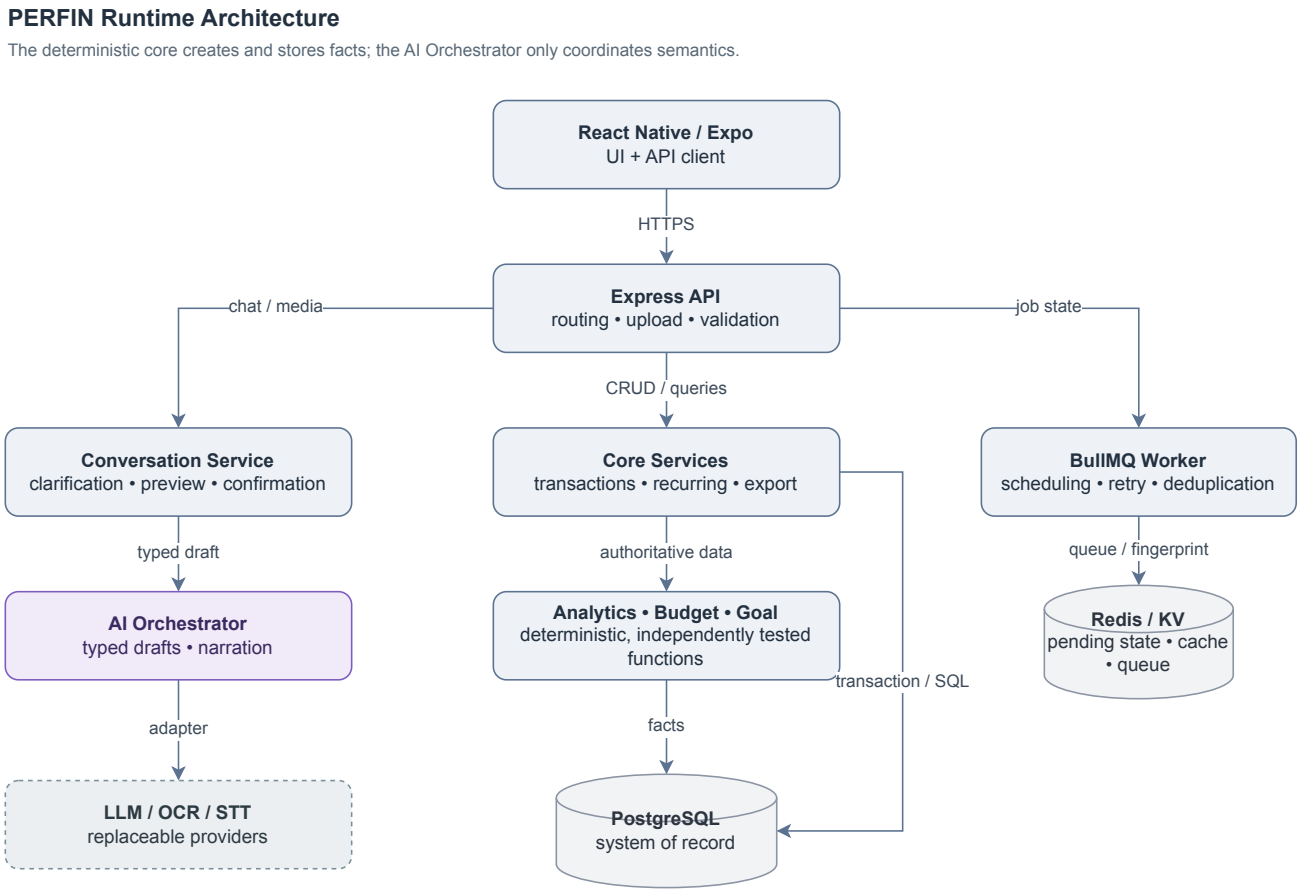

Figure 3: PERFIN runtime architecture; the deterministic core creates and stores facts while the AI Orchestrator coordinates semantics.

Table 23: Architectural components and responsibilities

Component	Responsible for	Not responsible for
Mobile App	Capture input, show previews/facts and confirm.	Database access or AI secrets.
Express Routes	HTTP contracts, upload and input validation.	Analytical formulas.
Core Services	Business rules, transactions and idempotency.	LLM-generated numbers.
Analytics, Budget, Goal	Deterministic facts and plans.	Arbitrary advice.
AI Orchestrator	Tool routing, extraction, narration and fallback.	Direct database writes.
PostgreSQL	System of record, constraints and aggregates.	Temporary conversation state.
Redis/BullMQ	TTL state, cache, queue and retry.	Financial source of truth.

Microservices were considered but not selected because prototype scale does not justify deployment, tracing and distributed-consistency costs. Serverless is also a poor fit for a long-running worker and local media providers. A modular monolith keeps database transactions simple and leaves a future extraction path when real load evidence exists.

3.2.2. Data Design

Figure 4: PERFIN UML domain class model separating ledger, planning, recurring and conversation/feedback aggregates.

`User` is the ownership root; `Wallet`, `Transaction` and `Category` form the ledger; `Budget`, `FinancialGoal` and recurring classes form planning; `ChatMessage` and `AIFeedback` preserve traceable interaction. Engines are not entities because they only read data and create facts.

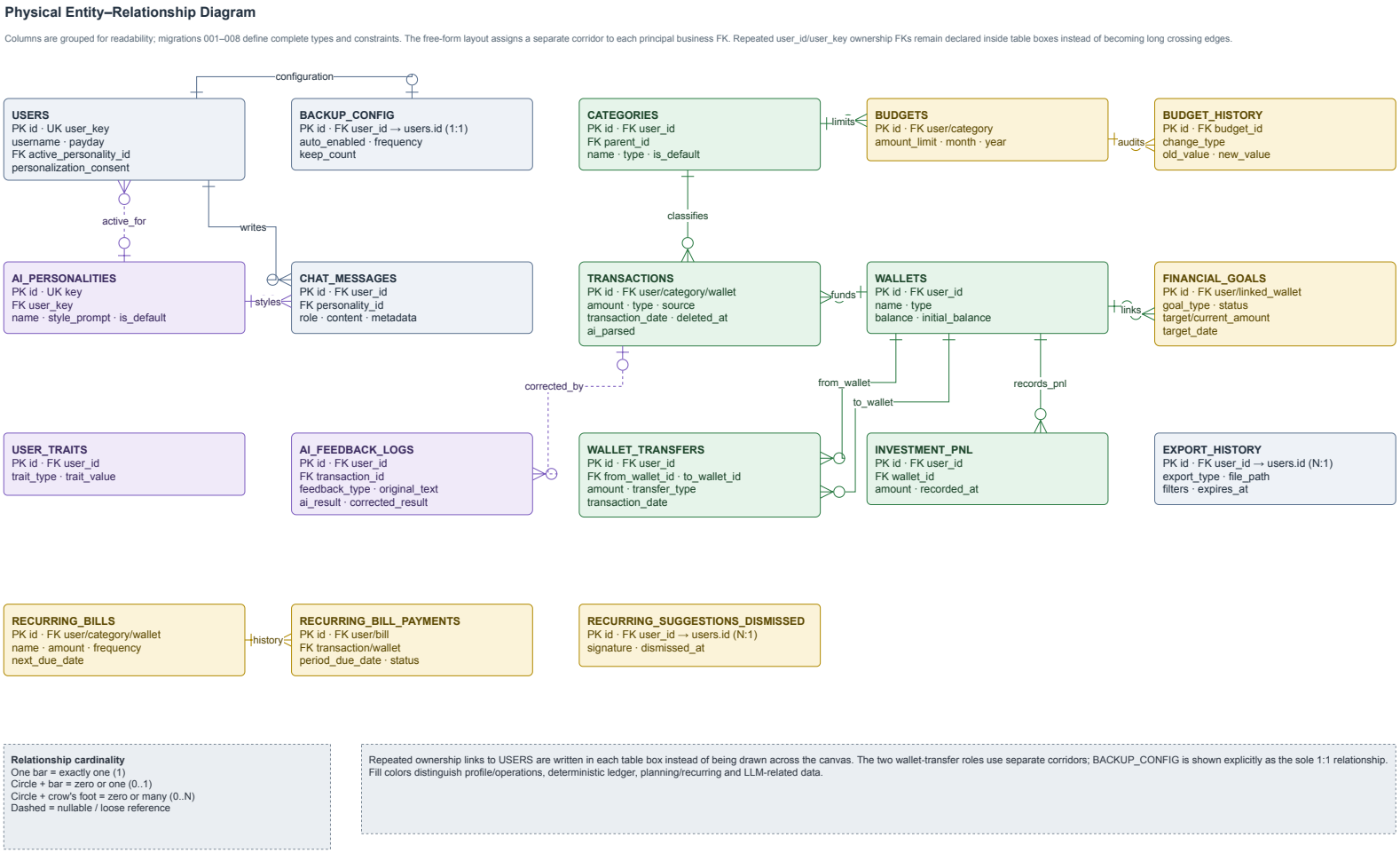

Figure 5: Physical ERD reconciled with runtime migrations; principal business foreign keys use separate orthogonal corridors.

Table 24: Entities in alphabetical order

Entity	Type	Description
ai_feedback_logs	History	Original AI result, correction and context.
ai_personalities	Catalogue	Selectable persona and style prompt.
backup_config	Configuration	Per-user backup policy; worker backup is not proven.
budget_history	History	Audit trail of budget-limit changes.
budgets	Planning	Category and monthly limits.
categories	Catalogue	Income/expense taxonomy and parent relation.
chat_messages	History	Conversation, facts/metadata and event key.
export_history	Operations	Exported files, filters and expiry.
financial_goals	Planning	Goal, progress, deadline and linked wallet.
investment_pnl	Ledger	Realized investment gain/loss.
recurring_bill_payments	Ledger	Payment record for each recurring period.
recurring_bills	Planning	Amount, frequency and next due date.
recurring_suggestions_dismissed	History	Signature of a dismissed recurring suggestion.
transactions	Ledger	Income/expense, input source and soft-delete status.
user_traits	Personalization	Consent-dependent user traits.
users	Data root	Demo profile and data-scope key.
wallet_transfers	Ledger	Transfer between two wallets.
wallets	Ledger	Wallet balance, type and name.

Money uses DECIMAL(15,2); report queries exclude soft-deleted records; deleting a referenced category is blocked or controlled by re-tagging; pending state exists only in Redis; and target-design ID queries include user_id/user_key.

3.2.3. Detailed Design

3.2.3.1. LLM Boundary and Input

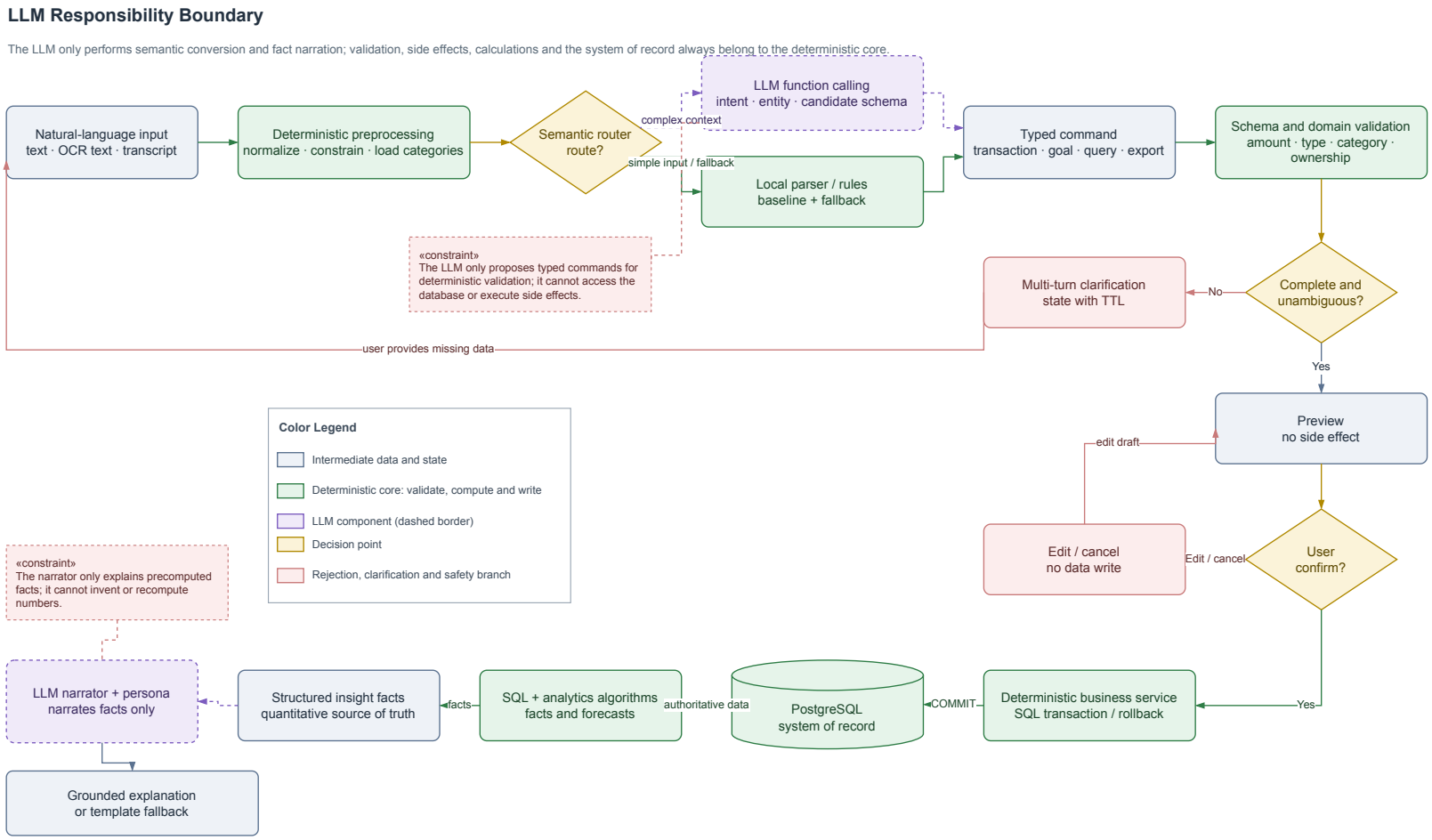

Figure 6: LLM responsibility boundary: typed drafts and narration remain outside validation, calculation and data-write authority.

The LLM only creates a typed command or narrates facts. The backend validates schema and domain rules, builds a preview and waits for confirmation before opening a transaction. For analysis, SQL and the engine create facts before narration. A parser or deterministic template provides a testable fallback.

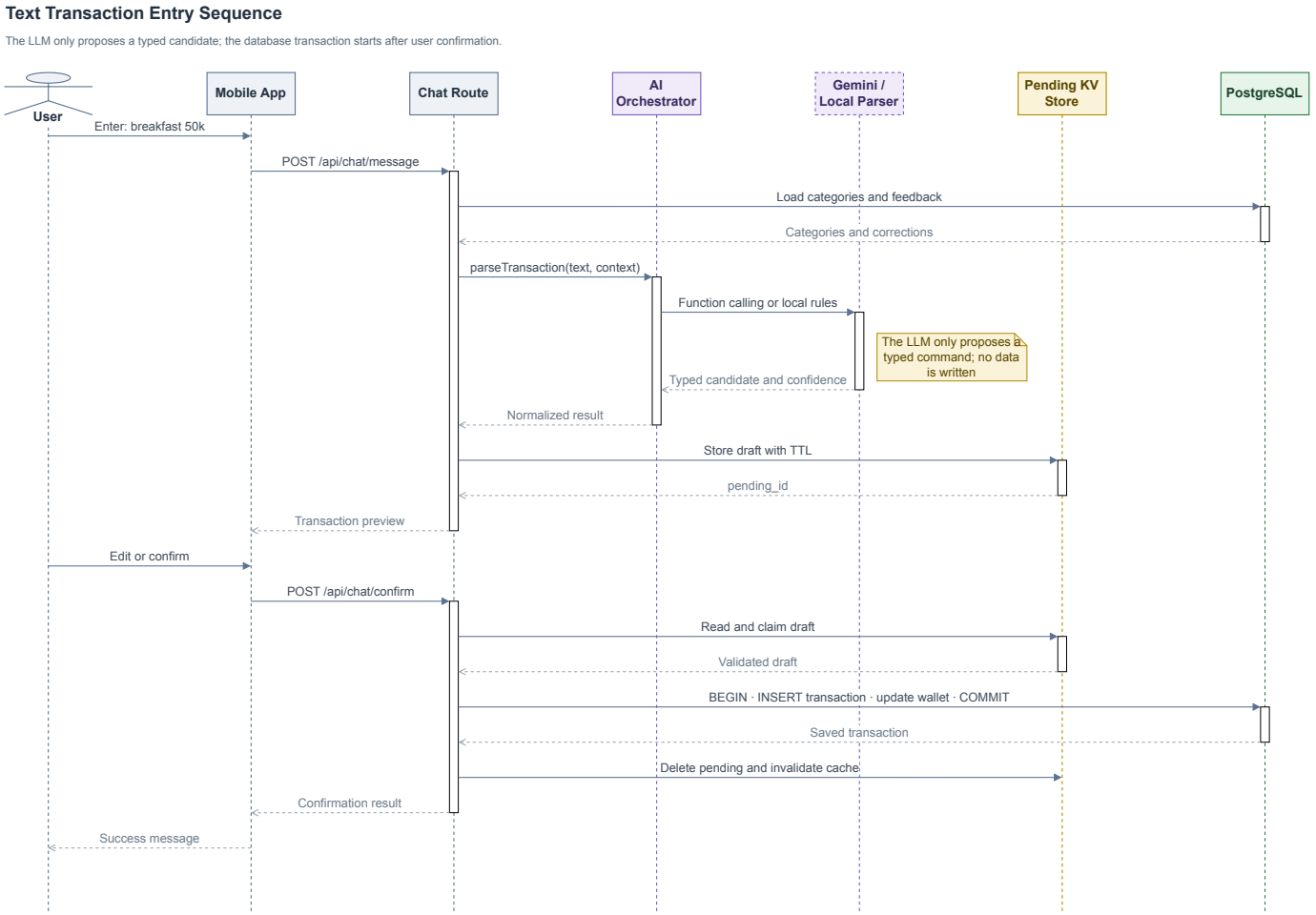

Figure 7: Text-entry sequence; the data-write transaction starts only after confirmation.

Parsing/preview has no side effect; confirm/commit atomically claims pending state. State contains intent, missing fields, candidates and a draft with a five-minute TTL. Concurrent confirmation must not create two transactions from one `pending_id`.

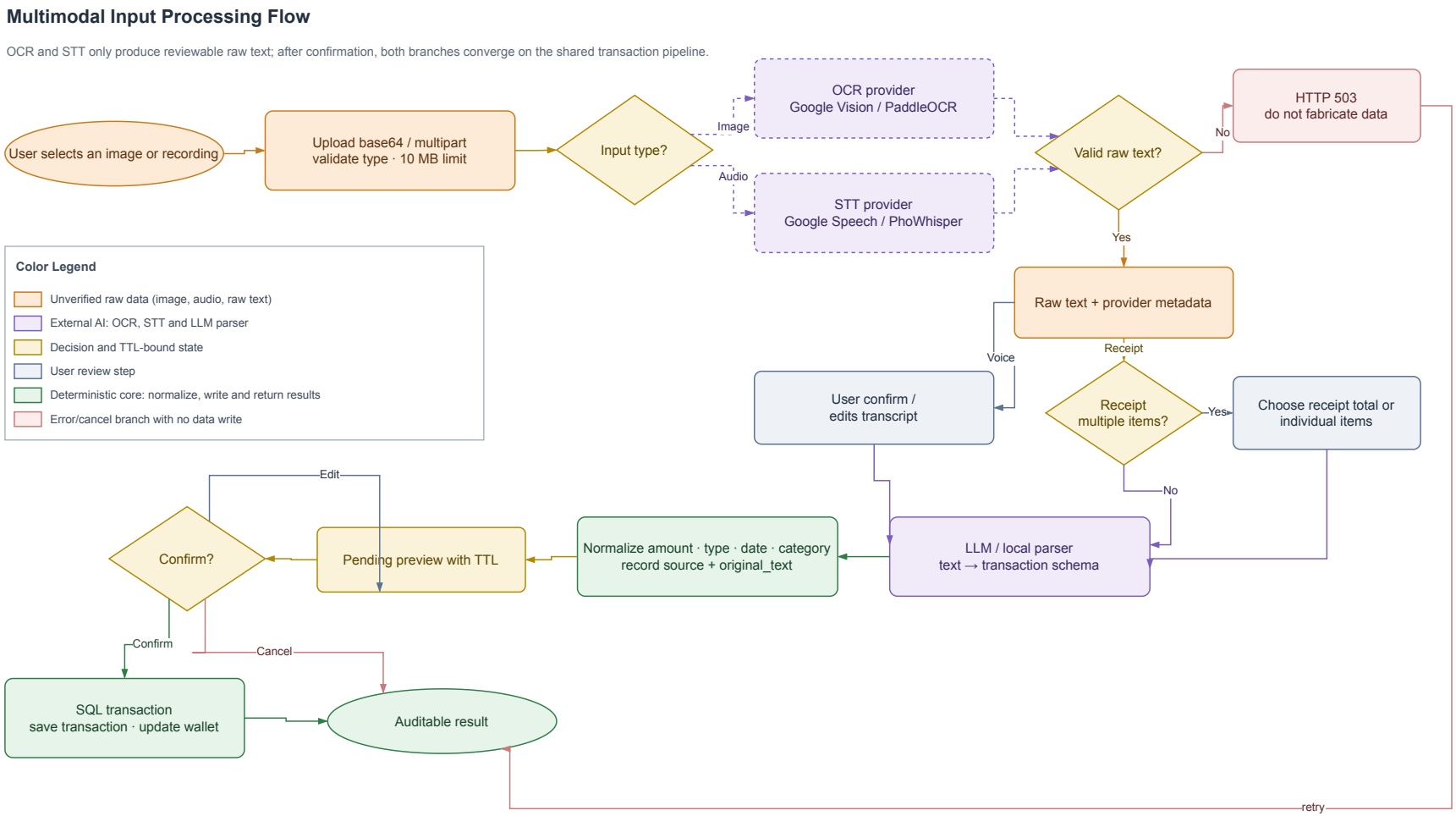

Figure 8: Image and speech processing; both branches converge on the text pipeline after raw-text review.

OCR/STT returns only raw text and provider metadata. Current ground truth is too small and receipt line totals are not automatically reconciled, so Figure 8 is a design flow, not evidence of accuracy.

3.2.3.2. Classification and Correction Retrieval

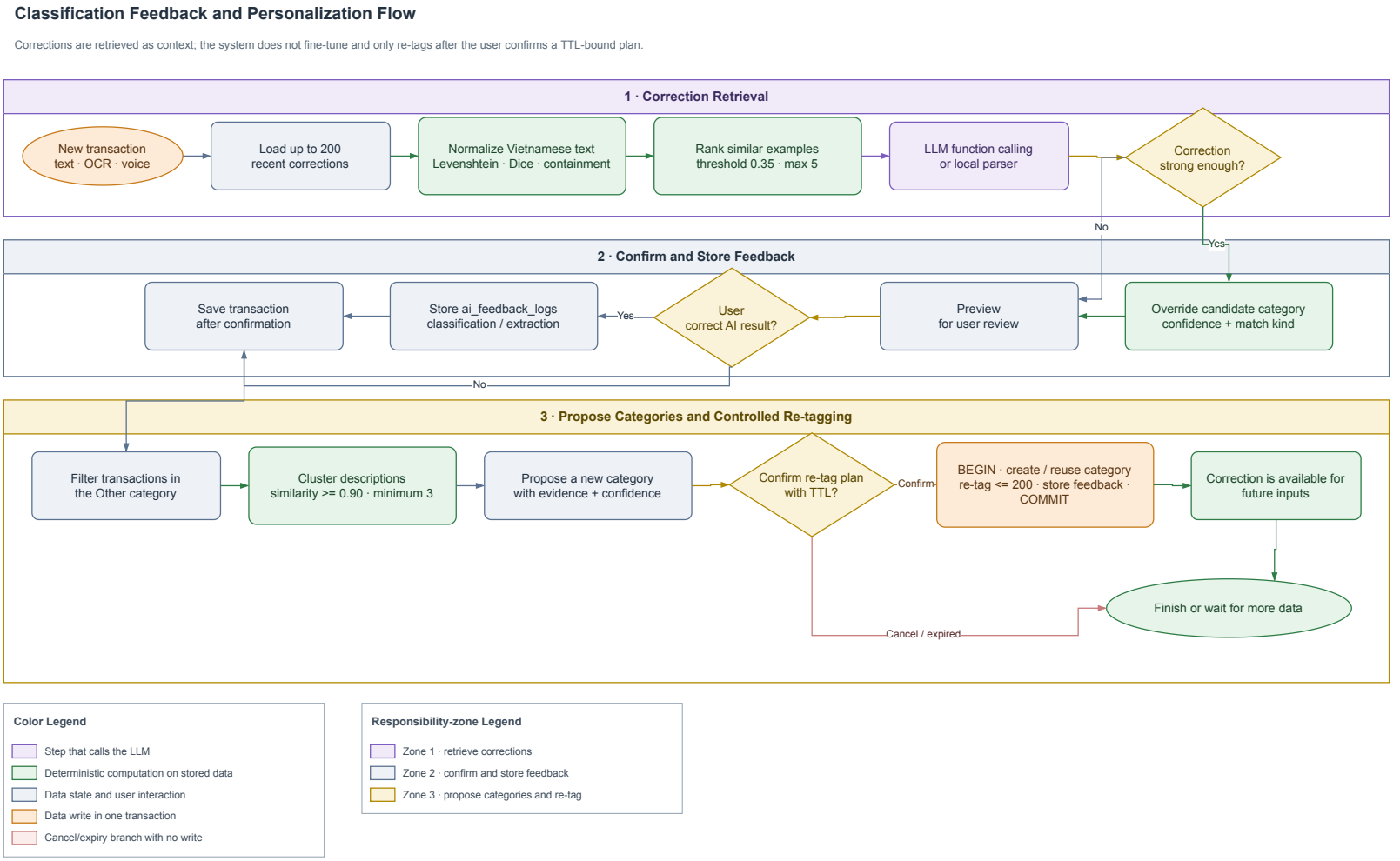

Figure 9: Classification and correction-retrieval flow; feedback is stored only after confirmation.

The matcher uses

$$s = \max(s_{edit},\ 0.92 s_{dice},\ s_{contain}). \tag{3.1}$$

Selection proceeds through exact name, exact alias and fuzzy matching. Long input uses $\tau = 0.82$, input of at most four characters uses $\tau = 0.90$, and the best candidate must exceed the second by $m = 0.08$. These are prototype starting values, not universal optima. Retrieval loads at most 200 recent corrections and selects at most five similar examples; conflict lowers confidence instead of forcing a label.

3.2.3.3. Analytics and Planning Algorithms

Default windows are 14 days for runway, 30 days for anomaly, 90 days for recurring discovery, six months for trend and 12 weeks for correlation. Every result reports its configured window.

For total balance B, daily expense e_j and $W = 14$,

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{3.2}$$

If $B \le 0$, then $d = 0$; if $\bar{e}_W = 0$, the depletion date is undefined. Recurring grouping uses mean amount $\bar{a}$, relative deviation $\delta_i = |a_i - \bar{a}|/\bar{a}$ and mean cadence. A group is stable when all $\delta_i \le 0.15$, cadence is 20–40 days or at least three occurrences exist.

For category spending $s_{c,j}$ across m months, a five-percent-buffer baseline is

$$b_c = 1.05 \left(\frac{1}{m}\sum_{j=1}^{m} s_{c,j} \right). \tag{3.3}$$

Month-end forecast is $\widehat{S}_D = (S/d_e)D$, where S is spending so far, d_e elapsed days and D month length. 50/30/20 and hybrid strategies shrink allocations when their sum exceeds the cap; recommendations require confirmation.

For remaining goal amount R and monthly contribution $M > 0$, $n = \lceil R/M \rceil$. For debt principal L, monthly rate r and duration n, level payment is

$$P = \frac{Lr}{1 - (1+r)^{-n}} \tag{3.4}$$

for $r > 0$, and $P = L/n$ for $r = 0$. A payment below first-month interest triggers a negative-amortization warning. What-if runs the same function with new parameters and does not modify the original plan.

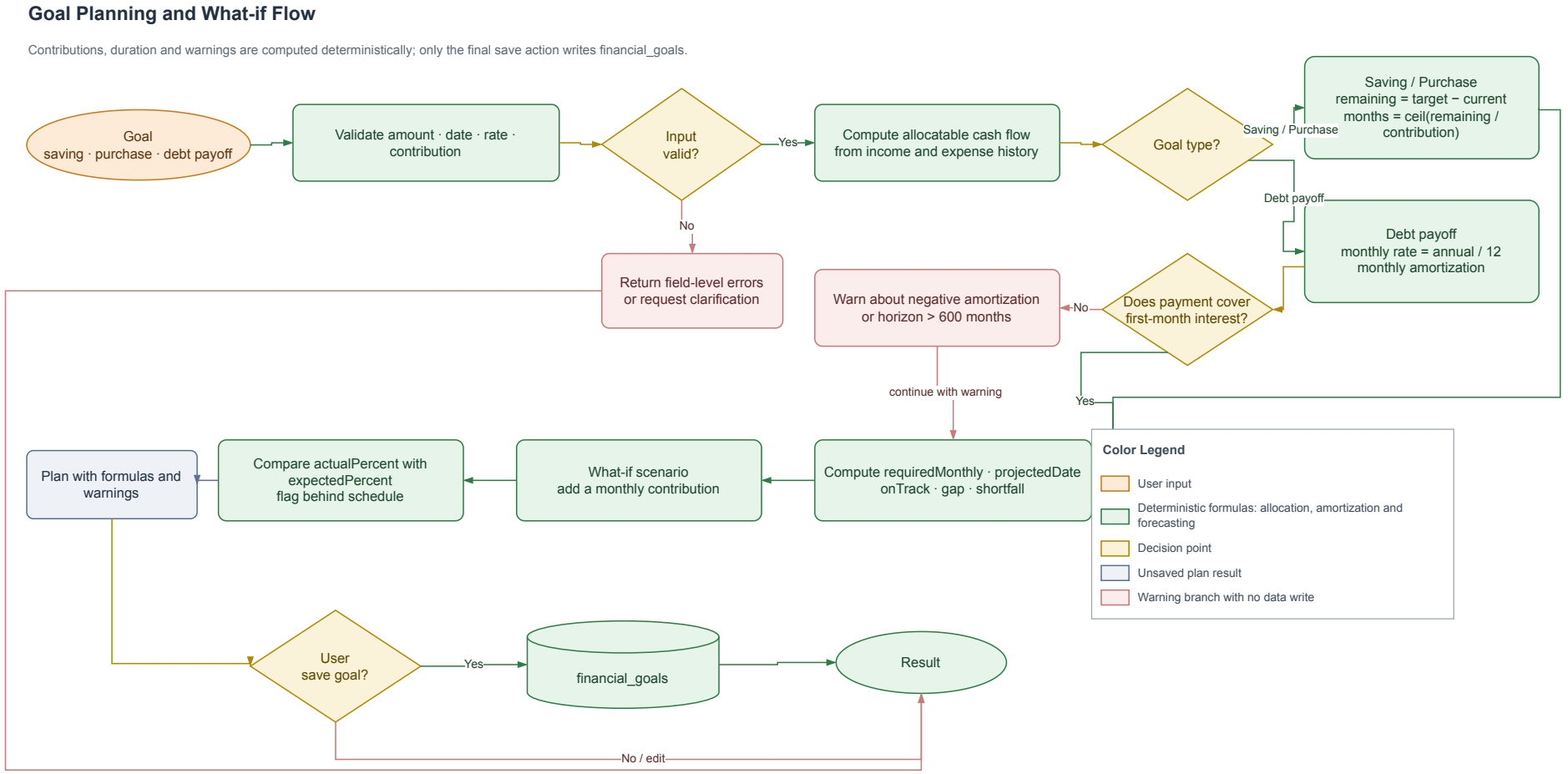

Figure 10: Goal planning and what-if flow; only final confirmation persists a goal.

3.2.3.4. Grounded Insight Generation

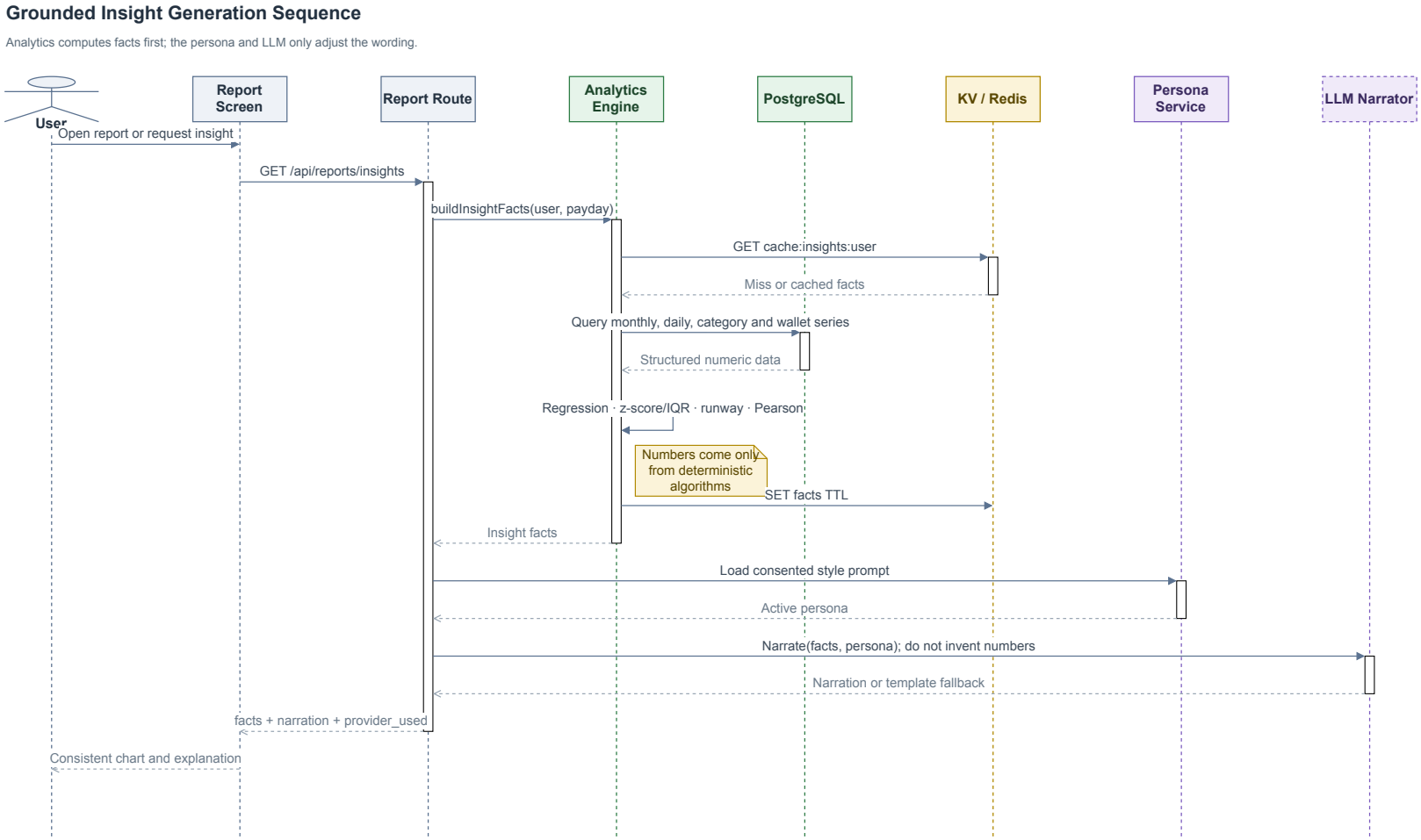

Figure 11: Grounded-insight sequence; Analytics returns facts before persona/LLM narration.

Insight facts include value, unit, window, sample count, method and warnings. A persona changes tone only with consent. The response includes both narration and facts for UI/test comparison. A numeric checker covering every expression remains a target design.

3.3. TESTING

3.3.1. Test Plan

Table 25: Test-plan matrix

Level	Target	Technique	Expected evidence
Unit	Matching, analytics, budget, goal and validation.	Normal, boundary, hand-computed expected values and invariants.	`node:test` pass/fail by case.
Integration	Route–service–model, PostgreSQL and Redis.	Test DB, rollback, TTL and concurrent claim.	Logs, DB snapshots and exit code.
System	Text/voice/image through preview and DB.	End-to-end scripted smoke.	HTTP response, DB assertion and UI image.
AI contract	Parser, tool schema and LLM.	Labelled inputs, same task and metrics.	JSON predictions, accuracy and macro-F1.
Media	OCR/STT adapters.	Ground-truth image/audio.	CER/WER and field accuracy.
Worker	Retry, deduplication and schedule.	Repeated fingerprint and fault injection.	Before/after effect count.
UAT	Target users.	Timed tasks and step count.	Completion, duration and SUS.

The text dataset must cover currency units, written numbers, multiplication, relative dates, multiple transactions, typos, mixed Vietnamese/English, missing or ambiguous fields and conflicting corrections. Analytics data needs known slopes, outliers, cadence and correlation. Media data must store image/audio with a reference transcript or receipt table.

3.3.2. Results and Analysis

3.3.2.1. Measured Operational Results

Table 26: Measured results and conclusion boundaries

Item	Result	Status	Permitted conclusion
Backend `npm test`	182/182 pass.	**Measured**	Unit/service tests across 38 files; not UAT or production evidence.
Local-parser gate	31/31 strict pass.	**Measured**	Fixed set of 31 sentences; not an independent benchmark.
Full API/DB/media smoke	23/23 checks pass.	**Measured**	Selected fixtures traverse HTTP–PostgreSQL/provider; Redis worker excluded.
Mobile-web UI smoke	4/4 gates pass at 390×844.	**Measured**	No overflow and images render; not UAT.
Data import	5,265 rows, 0 rejects.	**Measured**	Provenance-aware importer reconciled on demo PostgreSQL.
Parser on 5,265 rows	Accuracy 29.36%; macro-F1 0.177.	**Measured**	Historical labels conflict with current taxonomy; useful for error analysis.
Parser vs Gemini	Macro-F1 0.204 vs 0.607.	**Measured**	Same stratified 63-sentence sample; Gemini p50 964 ms.
Correction retrieval	Accuracy 68.19% on parser-wrong holdout.	**Measured**	Baseline 0% follows from sample selection; not a whole-system improvement.
OCR/STT accuracy	Not reported.	**Not measured**	Two images and one audio smoke fixture cannot establish CER/WER/field accuracy.

Item	Result	Status	Permitted conclusion
Live Redis worker	Not confirmed.	**Not measured**	Payment logic is tested, but live queue/worker was not measured.
Grounding, p95 and UAT	Not reported.	**Not measured**	Requires checker, at least 30 runs per flow and target users.

Smoke testing only establishes completion of selected scenarios. It is not OCR/STT accuracy evidence, and a tested payment handler does not prove that a live Redis worker ran.

3.3.2.2. Classification Experiments

Three experiments on 24 July 2026 use `dataFinance.csv` with 5,265 rows. Reproduction code and JSON/Markdown artifacts reside under `demo/backend/tests/experiments/` and `log/`.

Table 27: Local-parser result on 5,265 transactions

Metric	Value
Accuracy	29.36%
Macro-F1 (12 represented classes)	0.177
Weighted-F1	0.301
Incorrect cases	3,719/5,265
Errors involving Other	2,957

Most errors involve Other because some historical labels describe the funding source while the parser categorizes spending purpose. Taxonomy and labelling policy therefore need to be locked before model optimization.

Table 28: Local parser and Gemini ablation on the same 63 sentences

Branch	Accuracy	Macro-F1	p50	Note
Local parser	22.2%	0.204	0 ms	No API call.
Gemini	59.5%	0.607	964 ms	63/63 calls succeeded; 21 empty labels count as wrong.

Gemini performs better on this small sample but introduces latency and provider dependence. The result supports continued LLM extraction experiments; it does not show that one model is universally better for every user or taxonomy.

3.3.2.3. Correction-retrieval Experiment

After removing Other, the experiment selected 3,502 sentences the parser had already classified incorrectly, then split seed and holdout. Because selection guarantees parser error in the holdout, pre-correction accuracy is necessarily 0%. Table 29 measures errors repaired inside this selected set, not improvement over a representative baseline.

Table 29: Correction retrieval on a parser-wrong holdout

Metric	Before	After	Interpretation
Accuracy	0.00%	68.19%	1,194 selected errors repaired.
Macro-F1	0.000	0.544	Computed within the selected holdout.
Weighted-F1	0.000	0.782	Influenced by holdout class distribution.

The control group contains 1,330 parser-correct sentences; correction changed 466 incorrectly, 430 of which shared text with conflicting historical labels. Degradation on unambiguous sentences is 36/1,330 (2.7%). Retrieval can repair recurring errors but is label-noise sensitive; a future experiment needs random whole-distribution sampling and confidence intervals.

3.3.3. Requirements-to-Evidence Traceability

Table 30 links observable requirements to design, tests and evidence status. A route, mock or diagram alone is not end-to-end acceptance evidence.

Table 30: FR–design–test–evidence traceability

Req.	Design	Related test	Evidence and gap
FR-01, FR-03–FR-06	Typed draft, TTL preview, service and SQL transaction.	Parser gate, backend tests and API–DB smoke.	Selected fixtures and rollback invariants pass; no independent task-based user measurement.
FR-02	Media adapter, raw-text review and shared FR-01 pipeline.	Upload/media smoke; planned CER/WER and field accuracy.	Sample provider flow executes; insufficient ground truth for OCR/STT accuracy.
FR-07–FR-10	Analytics, facts contract, Budget/Goal service and grounded narrator.	Hand-expected unit tests, classification benchmark and planned checker.	Formulas and functions have tests; response-wide grounding and warning thresholds remain incomplete.
FR-11	Recurring service, BullMQ, event key and retry.	Payment/handler unit tests; planned live worker and fault injection.	Business logic is tested; no live queue/worker evidence in this evaluation.
FR-12	CSV exporter, history and TTL cleanup.	Escaping tests, API smoke and planned expiry check.	CSV has evidence; the former PDF-labelled path only generates HTML.
NFR-01–NFR-08	Constraints, user scope, fallback, idempotency and fact metadata.	Rollback, cross-scope, load, recovery, worker and UAT.	Test integrity has evidence; three-hour recovery, p95, UAT, live worker and faithfulness remain targets.

3.3.4. Threats to Validity

3.3.4.1. Internal Validity

Benchmarks depend on historical labels. Identical descriptions with different categories show that label noise can penalize both parser and retrieval even when a prediction follows the current taxonomy. The 63-sentence ablation is small, and the correction holdout is conditionally selected from parser errors, so observed differences are

not unbiased whole-traffic estimates. Gemini is version- and time-dependent; artifacts should retain model, parameters, timestamp and raw response.

3.3.4.2. Construct Validity

The 31/31 gate measures success on designed sentences, not out-of-sample correctness. Smoke tests measure scenario completion, not accuracy or usability. Accuracy can hide rare classes, so macro-F1, weighted-F1, per-class confusion and support are also needed. For OCR/STT, CER/WER alone is insufficient: amount, date and merchant field accuracy maps more directly to ledger risk.

3.3.4.3. External Validity

Data comes from one demo profile, uses VND and is primarily Vietnamese. Results do not automatically generalize to many users, other taxonomies, varied receipts or production load. The next evaluation should lock the taxonomy, sample randomly over time, split by normalized description, fix random seeds and add target-user UAT. Current conclusions apply only to the source, dataset and environment recorded in the 24 July 2026 artifacts.

CHAPTER 4: CONCLUSION

4.1. OBJECTIVE COMPLETION

PERFIN now forms a prototype with a clear separation between data, algorithms and the LLM layer. It contains 18 physical tables, transaction, analytics, feedback, budget, goal and state modules, and an automated test suite. Its central contribution is not replacing business logic with an LLM, but using the model as a language-conversion layer while the backend retains validation, computation and data-write authority.

Table 31: Objectives and evidence

Code	Main result	Conclusion
O1	Migrations, ERD, constraints and tested transactional flows.	Prototype-complete; broader DB fault injection is needed.
O2	Text/media adapters, typed draft, clarification, preview and confirmation.	Flow executes; OCR/STT accuracy is unmeasured.
O3	Trend, anomaly, runway, recurring, correlation, budget and goal planner.	Implemented with unit tests; thresholds need representative calibration.
O4	Tool schema, service validation, facts–narrator flow and fallback.	Boundary implemented; complete numeric faithfulness is unmeasured.
O5	182/182 backend tests, 23/23 full smoke and three classification experiments.	Evidence supports the demo core, not live worker, p95, UAT or production.

On 5,265 transactions, the local parser obtains macro-F1 0.177. In the same-sample 63-sentence ablation, parser and Gemini obtain 0.204 and 0.607. Correction retrieval repairs 68.19% of a holdout selected from sentences the parser had already misclassified. Its 0% baseline follows from selection, so the result demonstrates partial repair of repeated errors, not representative improvement over the whole distribution.

4.2. DELIVERABLES

Deliverables include mobile/backend source, PostgreSQL migrations, Redis/BullMQ configuration, LLM/OCR/STT adapters, tests, a reconciled demo dataset, experiment artifacts and this LaTeX report. The prototype supports text/media entry with review,

ledger management, analytics, budgets and goals, and returns facts with controlled narration.

The LLM handles free-form language better than the parser on the ablation sample, at the cost of latency and provider dependence. This value exists only behind four controls: schema, validation, preview/confirmation and fact grounding. PERFIN is therefore a financial-data system with an LLM interaction layer, not a chatbot that calculates or decides on the user's behalf.

4.3. LIMITATIONS

1. The prototype uses `default_user` and lacks production authentication and authorization.
2. Smoke tests on two images and one audio file cannot establish CER, WER or transaction-field accuracy.
3. Live Redis worker behaviour, retry/idempotency, p95 and UAT were not confirmed in a version-locked environment.
4. Fuzzy, anomaly, recurring and correlation thresholds and time windows are initial configurations rather than universal optima.
5. Historical taxonomy is inconsistent; similar text sometimes has different labels.
6. The numeric-grounding checker remains target design, and persona effects have not been evaluated.
7. Remaining gaps include true PDF export, full backup/restore, push notification and complete fresh-install wallet creation.

4.4. FUTURE WORK

Next priorities are to lock taxonomy and create an independent random evaluation set; build image/audio ground truth for CER/WER and field accuracy; run the Redis worker with fault injection and duplicate-effect checks; implement numeric grounding and measure p50/p95 over at least 30 runs per flow; conduct UAT comparing chat and forms; and add authentication, authorization, audit and secret management before real deployment.

Bank synchronization, shared wallets and high availability should follow only after data correctness and empirical evidence stabilize. This ordering preserves the basic-topic project's intended focus: data and algorithms are the core, while the LLM is a replaceable, measurable supporting component.

BIBLIOGRAPHY

[1] A. Lusardi and O. S. Mitchell, "The Economic Importance of Financial Literacy: Theory and Evidence," *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[2] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[3] L. R. Dice, "Measures of the Amount of Ecologic Association Between Species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[4] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed. Hoboken, NJ, USA: Wiley, 2012.

[5] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[6] K. Pearson, "Note on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.

[7] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. 9th International Conference on Document Analysis and Recognition*, vol. 2, pp. 629–633, 2007.

[8] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[9] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] Gemini Team, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.

[11] Google AI for Developers, "Function calling with the Gemini API," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling. [Accessed: 15-Jul-2026].

[12] PostgreSQL Global Development Group, "PostgreSQL Documentation — Transactions and Data Definition," 2026. [Online]. Available: https://www.post

gresql.org/docs/current/tutorial-transactions.html and https://www.postgresql.org/docs/current/ddl.html. [Accessed: 15-Jul-2026].

[13] Redis Ltd., "EXPIRE," *Redis Commands Documentation*, 2026. [Online]. Available: https://redis.io/docs/latest/commands/expire/. [Accessed: 15-Jul-2026].

[14] Taskforce.sh, "BullMQ Documentation: Idempotent Jobs and Retrying Failing Jobs," 2026. [Online]. Available: https://docs.bullmq.io/patterns/idempotent-jobs and https://docs.bullmq.io/guide/retrying-failing-jobs. [Accessed: 15-Jul-2026].

[15] Money Lover, "Money Lover — Money Manager, Budget Expense Tracker," 2026. [Online]. Available: https://moneylover.me. [Accessed: 24-Jul-2026].

[16] MISA JSC, "MISA MoneyKeeper — Personal Expense Management Application," 2026. [Online]. Available: https://www.misa.vn/moneykeeper. [Accessed: 24-Jul-2026].

[17] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

CHAPTER A: INSTALLATION GUIDE

The minimum environment is Node.js/npm, PostgreSQL and Expo Go or a browser. Redis 7 is recommended for the worker. Without Redis, the API may use an in-memory KV fallback during development, but queue and retry behaviour is not demonstrated.

A.1. BACKEND SETUP

From the project root, copy the sample configuration, supply PostgreSQL settings and retain `AI_PROVIDER=local` when no Gemini key is used. Never commit secrets or service-account files.

```
cd demo/backend
cp .env.example .env
npm install
npm run migrate
npm start
```

The API defaults to http://127.0.0.1:3000. Use that URL for a basic availability check. Recreate schema and seed data only on a development/test database:

```
npm run migrate:fresh
```

A.2. REDIS AND WORKER

From `demo`, start Redis with Docker Compose and open another terminal for the worker:

```
docker compose up -d redis
cd backend
npm run worker
```

Verify `REDIS_URL`, `JOBS_ENABLED` and timezone in `backend/.env`. `docker compose down` stops Redis while retaining its volume.

A.3. FRONTEND SETUP

```
cd demo/frontend
npm install
EXPO_PUBLIC_API_URL=http://127.0.0.1:3000 npm start
```

For a phone on the same LAN, replace `127.0.0.1` with the backend host's LAN address and use `npm run start:lan`. If the device cannot reach that address, use Expo tunnel plus a separate backend tunnel. Never expose AI secrets through frontend environment variables.

CHAPTER B: QUICK USER GUIDE

1. Open the application and check that Dashboard shows balance, total income/expense and recent transactions.
2. Add a transaction from Transactions, or open Chat and enter a sentence such as "ăn phở 50k sáng nay".
3. For image/audio, review and edit the OCR text or transcript. Media results are not stored automatically.
4. Review amount, type, date, category and wallet in the preview; edit, cancel or confirm. Only confirmation creates a transaction.
5. Use Budget for limits, Reports for trends/insights and Goals for plans and what-if scenarios.
6. Correct an incorrect category. Confirmed corrections may be retrieved for a later input.

The prototype has no login and uses `default_user`. Do not enter sensitive data or treat results as guaranteed investment advice.

CHAPTER C: REPRODUCTION COMMANDS

Run each command from the named component directory. Record commit, Node version, provider configuration and PostgreSQL/Redis state without storing secrets in artifacts.

```
cd demo/backend
npm test
npm run test:ai
npm run smoke:full

cd ../frontend
npm run ui:smoke
npx expo export --platform web --output-dir /tmp/perfin-web
```

Media smoke tests require a real provider and fixtures. Without ground truth, report availability pass/fail only. Integration tests need a separate test database and cleanup after each group.

CHAPTER D: DIAGRAM SOURCES

The report keeps 14 editable Draw.io sources and one overall use case. Every diagram label is English so the Vietnamese and English reports share the same set. Sources are in `latex/figures/drawio/`; PDF/PNG/SVG exports are in `latex/figures/rendered/`.

No.	Subject	Basename
1	System context	`01-system-context`
2	Runtime architecture	`02-runtime-architecture`
3	Prototype deployment	`03-deployment`
4	Domain class diagram	`04-domain-class`
5	Physical ERD	`05-physical-erd`
6	LLM boundary	`06-llm-boundary`
7	Conversation state	`07-conversation-state`
8	Text transaction sequence	`08-text-sequence`
9	Multimodal input	`09-multimodal-flow`
10	Feedback and classification	`10-feedback-flow`
11	Grounded insight	`11-insight-sequence`
12	Goal planner and what-if	`12-goal-flow`
13	Background job	`13-worker-sequence`
14	Overall use case	`14-usecase-overview`